

Dokumentacja: Implementacja Wielomianów Gęstych na Bazie List Pythona

Anna Dębska

10.01.2025

Spis treści

1 Wprowadzenie	3
2 Opis problemu	3
3 Opis Interfejsu	3
3.1 Tworzenie wielomianu	3
3.2 Wyświetlanie wielomianu	3
3.3 Odczyt stopnia wielomianu	3
3.4 Sprawdzanie, czy wielomian jest zerowy	4
3.5 Dodawanie wielomianów	4
3.6 Odejmowanie wielomianów	4
3.7 Mnożenie wielomianów	4
3.8 Obliczanie wartości wielomianu w punkcie	4
3.9 Porównywanie wielomianów	4
3.10 Dostęp do współczynnika przy danej potęgze	4
4 Uwagi na Temat Implementacji	5
5 Analiza Złożoności Algorytmicznej	6
6 Podsumowanie i Wyniki Testów	6
7 Literatura	7

1 Wprowadzenie

Niniejszy dokument opisuje implementację **wielomianów gęstych** na bazie **list Pythona**. Klasa `Polynomial` umożliwia reprezentację i wykonywanie operacji na wielomianach, których współczynniki odpowiadające kolejnym stopniom wielomianu są przechowywane w formie list. Projekt zawiera zaawansowane metody manipulacji wielomianami oraz funkcje wspierające testowanie poprawności działania kodu.

2 Opis Problemu

Wielomiany (ang. *Polynomials*) są powszechnie stosowanymi obiektami matematycznymi w wielu dziedzinach, takich jak algebra, analiza matematyczna czy numeryczne przetwarzanie danych. Problemem jest efektywne zaimplementowanie operacji matematycznych, takich jak:

- Dodawanie, odejmowanie i mnożenie wielomianów
- Porównywanie wielomianów
- Obliczanie wartości wielomianu dla podanej zmiennej x
- Porównywanie wielomianów
- Reprezentacja wielomianu w przyjaznej formie tekstowej.

Wymagania obejmują również zachowanie niskiej złożoności czasowej i pamięciowej oraz łatwość testowania i rozszerzania implementacji.

3 Opis Interfejsu

3.1 Tworzenie wielomianu

Konstruktor klasy `Polynomial` umożliwia inicjalizację wielomianu na podstawie listy współczynników:

```
p = Polynomial([5, -2, 3])    # 5 - 2x + 3x^2
```

3.2 Wyświetlanie wielomianu

```
print(Polynomial([3, 0, -1])) # wyświetla: -x^2 + 3
```

3.3 Odczyt stopnia wielomianu: `degree`

<code>p.degree()</code>	<code># np. dla [5, -2, 3]: 2</code>
-------------------------	--------------------------------------

3.4 Sprawdzanie, czy wielomian jest zerowy: `is_zero`

Metoda zwraca True, jeśli wszystkie współczynniki wielomianu są zerami

<code>p.is_zero()</code>	<code># np. dla [0, 0, 0]: True</code>
--------------------------	--

3.5 Dodawanie wielomianów +

<code>p1 + p2</code>	<code># dodaje współczynniki odpowiadających sobie potęg</code>
----------------------	---

3.6 Odejmowanie wielomianów -

<code>p1 - p2</code>	<code># odejmuje współczynniki odpowiadających sobie potęg</code>
----------------------	---

3.7 Mnożenie wielomianów *

<code>p1 * p2</code>	<code># wynik to nowy wielomian o stopniu równym sumie stopni p1 i p2</code>
----------------------	--

3.8 Obliczanie wartości wielomianu w punkcie: `__call__`

<code>p(2)</code>	<code># oblicza wartość wielomianu dla x = 2</code>
-------------------	---

3.9 Porównywanie wielomianów `==` / `!=`

Operator `==` sprawdza, czy dwa wielomiany są równe. Wielomiany są uznawane za równe, jeśli ich różnica daje wielomian zerowy. Operator `!=` sprawdza, czy wielomiany nie są równe.

<code>p1 == p2</code>	<code># True, jeśli p1 i p2 są równe</code>
-----------------------	---

<code>p1 != p2</code>	<code># True, jeśli p1 i p2 nie są równe</code>
-----------------------	---

3.10 Dostęp do współczynnika przy danej potędze `[]`

<code>p[2]</code>	<code># zwraca współczynnik przy x^2</code>
-------------------	---

<code>p[100]</code>	<code># zwraca 0, jeśli współczynnik nie istnieje</code>
---------------------	--

Program można uruchomić za pomocą polecenia:

```
python main.py
```

4 Uwagi na Temat Implementacji

Implementacja opiera się na wykorzystaniu list w celu przechowywania współczynników wielomianu. Kolejne wartości w listach reprezentują kolejne współczynniki zaczynając od stopnia najniższego. Klasa zawiera metody optymalizujące działanie, takie jak usuwanie nadmiarowych zer, które mogą pojawić się na końcu listy współczynników. To pozwala na oszczędność pamięci oraz uproszczenie operacji.

```
# konstruktor: przypisywanie współczynników od najniższego stopnia do wielomianu
def __init__(self, coefficients):  # Anna Dębska *
    if not isinstance(coefficients, list) or not all(isinstance(c, (int, float)) for c in coefficients):
        raise ValueError("Coefficients must be a list of numbers.")
    self.coefficients = coefficients
    self._remove_unnecessary_zeros()

# usuwanie niepotrzebnych zer na końcu wielomianu
def _remove_unnecessary_zeros(self):  # usage  # Anna Dębska *
    while len(self.coefficients) > 1 and self.coefficients[-1] == 0:
        self.coefficients.pop()
```

Konstruktor i edycja współczynników

Metoda pozwala przekształcić równanie z np. „ $1 + 3x + 7x^2 + 0x^3 + 0x^4 + 0x^5$ ” na „ $1 + 3x + 7x^2$ ”.

Do obliczania wartości wielomianu w danym punkcie zastosowano algorytm Hornera, który redukuje liczbę operacji mnożenia i dodawania.

```
# obliczanie wartości wielomianu dla podanej wartości x
def __call__(self, x):  # Anna Dębska *
    if not isinstance(x, (int, float)):
        raise TypeError("Argument must be a number.")
    result = 0
    for coeff in reversed(self.coefficients):
        result = result * x + coeff
    return result
```

Algorytm Hornera

5 Analiza Złożoności Algorytmicznej

<code>__init__</code>	$O(n)$
<code>_remove_unnecessary_zeros</code>	$O(n)$
<code>degree</code>	$O(1)$
<code>is_zero</code>	$O(n)$
<code>__eq__</code>	$O(n)$
<code>__ne__</code>	$O(n)$
<code>__str__</code>	$O(n)$
<code>__getitem__</code>	$O(1)$
<code>__add__</code>	$O(n)$
<code>__sub__</code>	$O(n)$
<code>__mul__</code>	$O(n*m)$
<code>__call__</code>	$O(n)$

Gdzie n i m to stopnie wielomianów, $n > m$

6 Podsumowanie i Wyniki Testów

Projekt oferuje efektywne i przejrzyste narzędzie do pracy z wielomianami, zapewniając wydajność operacji, czytelność kodu i łatwość użytkowania. Implementacja uwzględnia najważniejsze operacje algebraiczne oraz optymalizacje, takie jak algorytm Hornera. Dzięki obecności testów możliwe jest bezproblemowe rozszerzanie funkcjonalności.

Projekt zawiera moduł `unittest` do testowania funkcjonalności klasy `Polynomial`. W pliku `test.py` zawarto następujące testy:

- Sprawdzanie, czy konstruktor odpowiednio zgłasza błędne dane
- Sprawdzenie operacji dodawania, odejmowania i mnożenia
- Sprawdzenie operatorów `==` i `!=`
- Sprawdzenie poprawności wyświetlania wielomianów
- Sprawdzenie poprawności odczytu współczynników i stopnia wielomianu
- Sprawdzenie poprawności wartościowania (algorytmu Hornera)

Testy można uruchomić za pomocą polecenia:

```
python test.py
```

Wszystkie testy zakończyły się pomyślnie, co potwierdza poprawność i niezawodność implementacji.

7 Literatura

- "Introduction to Algorithms" - T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein.
- Python Documentation: <https://docs.python.org/3/>.
- Kursy akademickie z algebry